

NAME: KRISHNAPRIYA P

REG NO: 192511279

COURSE CODE: CSA0613

COURSE NAME: Design Analysis of Algorithm

CO4 – AT2 – Industry Problem Solving Task

Q20. Autonomous Vehicle Path Planning Optimization (Graph Algorithms)

1. Real-World Problem

An autonomous vehicle must compute the safest and shortest path from its current location to a destination in a dynamic environment. Unlike a normal shortest-distance problem, the vehicle must continuously consider obstacles, traffic, road conditions, travel time and safety. The route that is shortest at one moment may become unsafe or slow because of an accident, construction, congestion or a moving obstacle.

This problem can be modelled using a weighted graph. Road intersections, lane points or navigation locations are represented as vertices, while road segments are represented as edges. The objective is to find a path with minimum overall cost while satisfying safety and environmental constraints.

2. Graph Representation

The road network can be represented as:

$$G = (V, E)$$

- V represents road intersections, lanes or navigable points.
- E represents road segments connecting two points.
- Each edge has a weight representing distance, travel time, traffic penalty or safety risk.
- Blocked or unsafe road segments can be removed from the graph or assigned a very high cost.

For autonomous driving, the graph can be updated whenever the vehicle receives new information from cameras, LiDAR, radar, GPS, traffic services or the digital map.

3. Major Constraints

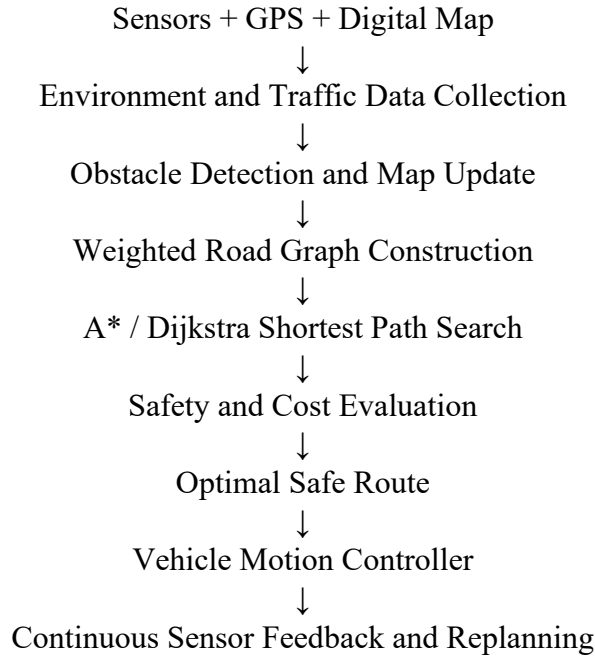
- Static obstacles: barriers, construction zones, parked vehicles and closed roads.
- Dynamic obstacles: pedestrians, cyclists and moving vehicles.
- Traffic conditions: congestion can increase travel time on a road.
- Road closures and weather: accidents, flooding or poor visibility may make an edge unavailable.
- Safety requirements: the vehicle must avoid high-risk roads and maintain safe distances.
- Real-time updates: the route must be recalculated when the environment changes.
- Computational limits: the algorithm must produce decisions quickly enough for real-time driving.

4. Shortest Path Graph Algorithms

- Dijkstra's Algorithm: finds the minimum-cost path when all edge weights are non-negative. It is reliable but may explore many unnecessary nodes.
- A* Algorithm: combines the cost already travelled with a heuristic estimate of the remaining distance to the destination. It is generally faster for point-to-point navigation.
- D* / D* Lite: designed for changing environments and can efficiently update a route when parts of the map change.

For an autonomous vehicle, A* is a strong choice for global point-to-point path planning, while dynamic replanning techniques are used when new obstacles or traffic information are detected.

5. Proposed System Design



6. Solution Workflow

Step 1: Collect Environment Data

The system receives the vehicle's current position and destination from GPS/navigation data. It also collects information about road geometry, speed limits, traffic density, weather and obstacles detected by vehicle sensors.

Step 2: Construct the Road Graph

The navigation map is converted into a graph. Each intersection or lane point becomes a node. A road segment connecting two nodes becomes an edge. The graph may contain thousands or millions of nodes for a large city, so efficient data structures are required.

Step 3: Assign Edge Costs

Each road segment receives a cost based on the factors important to the vehicle. A useful combined cost is:

$$\text{Cost}(\text{edge}) = \alpha \times \text{Distance} + \beta \times \text{Travel Time} + \gamma \times \text{Risk} + \delta \times \text{Traffic Penalty}$$

Here, α , β , γ and δ are weights that determine the importance of distance, time, safety and traffic. Therefore, the algorithm does not simply choose the physically shortest road; it chooses the route with the best overall cost.

Step 4: Detect and Handle Obstacles

If a road is blocked, the corresponding edge can be temporarily removed from the graph. If a road is unsafe but still usable, its cost can be increased. Moving obstacles are handled by updating the local environment representation as new sensor observations arrive.

Step 5: Find the Shortest and Safest Path

A* starts at the vehicle's current position and explores promising nodes toward the destination. Its evaluation function is:

$$f(n) = g(n) + h(n)$$

- $g(n)$ is the actual cost from the starting point to node n .

- $h(n)$ is the estimated cost from node n to the destination.
- $f(n)$ is the total estimated cost used to prioritize nodes.

When a suitable heuristic is available, A* can avoid exploring large parts of the graph that are unlikely to produce the best route.

Step 6: Validate the Route

Before execution, the selected path is checked against obstacle information, safety constraints and road availability. If a segment becomes invalid, the route is not followed blindly; the system starts a new planning cycle.

Step 7: Execute and Continuously Monitor

The vehicle follows the selected path using its motion-control system. At the same time, sensors continuously monitor the surroundings. This creates a feedback loop between perception, planning and vehicle control.

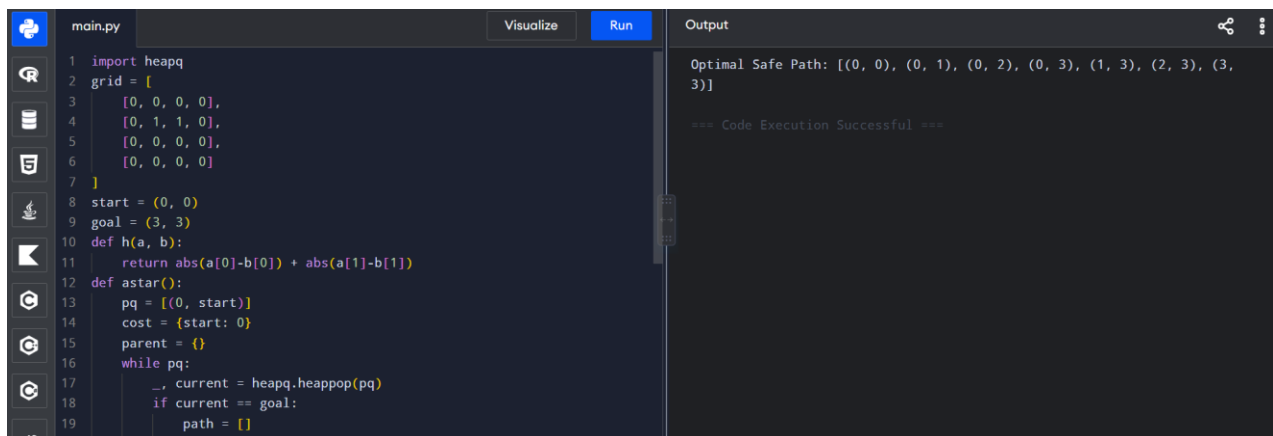
Step 8: Dynamic Replanning

Suppose an accident suddenly blocks the next road. The vehicle detects the change, updates the graph and recalculates the route. If traffic later becomes heavy on the alternative route, the edge cost changes again and another route can be selected. This adaptability is essential for real-world autonomous driving.

7. Example Scenario

Consider a vehicle travelling from a residential area to an airport. The shortest route initially passes through a major junction. A traffic accident occurs at that junction. The affected road segment is marked unavailable or assigned a very high cost. A* then evaluates alternative roads and selects a safe route. Although the new route may be slightly longer, it can have a lower total cost because it avoids the blocked and high-risk segment.

SOURCE CODE AND OUTPUT:



The screenshot shows a code editor with a file named 'main.py'. The code implements a grid-based A* search algorithm. The grid is a 6x6 array where 0 represents a free cell and 1 represents an obstacle. The start is at (0, 0) and the goal is at (3, 3). The heuristic function $h(a, b)$ is the Manhattan distance. The `astar()` function returns the optimal path. The output window shows the optimal path and a success message.

```

1 import heapq
2 grid = [
3     [0, 0, 0, 0],
4     [0, 1, 1, 0],
5     [0, 0, 0, 0],
6     [0, 0, 0, 0]
7 ]
8 start = (0, 0)
9 goal = (3, 3)
10 def h(a, b):
11     return abs(a[0]-b[0]) + abs(a[1]-b[1])
12 def astar():
13     pq = [(0, start)]
14     cost = {start: 0}
15     parent = {}
16     while pq:
17         _, current = heapq.heappop(pq)
18         if current == goal:
19             path = []

```

Output:

```

Optimal Safe Path: [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (3, 3)]
=== Code Execution Successful ===

```

8. Scalability and Adaptability

- Small road networks can be solved quickly using Dijkstra or A*.
- A* is suitable for large point-to-point navigation because its heuristic focuses the search toward the destination.
- Hierarchical maps can divide planning into levels, such as city roads, major roads and local lanes, reducing the search space.
- Dynamic replanning allows the system to respond to accidents, road closures, traffic and moving obstacles.
- Local planning can be combined with global graph search so that the vehicle handles nearby obstacles without recalculating the entire city map.

- Parallel or cloud-assisted computation can support large-scale route services, while safety-critical local decisions should remain available on the vehicle.

9. Computational Efficiency

For a graph with $|V|$ vertices and $|E|$ edges, Dijkstra's algorithm using a binary heap has approximately $O((|V| + |E|) \log |V|)$ time complexity. A* has a similar worst-case graph-search bound, but a good heuristic can reduce the number of nodes explored in practical point-to-point navigation.

The memory requirement depends on the number of vertices, edges and search states stored. For very large road networks, storing and processing the entire graph can be expensive. Therefore, practical systems use techniques such as hierarchical maps, graph partitioning, efficient priority queues and limited local replanning.

Computational efficiency is especially important because an autonomous vehicle cannot wait for a long offline calculation. The planning algorithm must provide a valid route within a short time and must be capable of updating it repeatedly.

10. Evaluation of the Proposed Solution

- Correctness: Dijkstra and A* can produce minimum-cost paths when their assumptions are satisfied.
- Speed: A* can be significantly more efficient than uninformed search for point-to-point navigation when the heuristic is effective.
- Adaptability: dynamic graph updates allow the route to respond to new obstacles and traffic conditions.
- Safety: assigning higher costs to risky roads allows safety to influence route selection.
- Scalability: hierarchical and local planning methods help manage large road networks.
- Limitation: extremely dynamic scenes and very large graphs can still require substantial computation and frequent updates.

11. Real-World Autonomous Driving Interpretation

In a real autonomous vehicle, graph-based planning is one layer of a larger driving system. The global planner determines a suitable route through the road network, while perception systems detect obstacles and the local planner handles immediate vehicle movement. When sensor data indicates that the environment has changed, the planner updates the graph and searches for another path.

For example, if the vehicle is approaching an intersection and a pedestrian or stopped vehicle blocks its planned lane, the system can mark that local path as unavailable and generate a safe alternative. Similarly, if live traffic information shows that another road has become congested, its travel-time cost can be increased. The final route therefore reflects the current environment rather than a fixed precomputed path.

12. Advantages

- Provides systematic shortest-path optimization.
- Can combine distance, time, traffic and safety into one cost model.
- Supports obstacle avoidance through graph updates.
- A* provides efficient goal-directed search.
- Can be adapted to changing environments using dynamic replanning.
- Can scale to large maps with hierarchical and efficient graph-processing methods.